School Management Database - User Manual

Version 1.0 | PostgreSQL 9.2 | Schema: `school` | Script: `school_management_postgres92.sql`

1. Introduction

This manual explains how to use the School Management database. The database stores everything a school needs to run day to day: users and roles, classes and sections, subjects, classrooms, timetables, student and faculty attendance, leave requests, homework, assignments, exams, marks and progress reports.

The database supports three roles:

Role	What they do in the system
Admin	Sets up classes, subjects, timetables and exams. Views attendance, exam and progress reports. Approves or rejects leave requests.
Faculty	Teaches assigned subjects and sections. Marks student attendance, assigns homework and assignments, enters marks, applies for leave.
Student	Views own course details, timetable, attendance, homework, assignments, exam schedule, marks and progress report. Applies for leave.

Note: the `roles` table controls what a person may do inside the school application. It is separate from PostgreSQL login roles (see Section 11.4).

2. Getting Started

2.1 Load the database

Open psql, connect to your database and run the script:

```
psql -U postgres -d school_db
\i 'C:/Users/yourname/Downloads/school_management_postgres92.sql'
```

Use forward slashes in the file path. The script drops and recreates the `school` schema, so running it again resets all data to the sample records.

2.2 Set the search path

All objects live in the `school` schema. Either prefix every table (`school.users`) or set the search path once per session:

```
SET search_path TO school;
```

2.3 Verify the installation

```
\dt school.*
SELECT COUNT(*) FROM users;
SELECT * FROM v_pending_leaves;
```

You should see 29 tables, 8 users and one pending leave request.

2.4 Useful psql commands

Command	Purpose
`\dt school.*`	List all tables
`\d students`	Show columns, keys and indexes of a table
`\dv school.*`	List all views
`\x`	Toggle expanded (vertical) output
`\q`	Quit psql

3. Database Overview

The database has 29 tables in 12 modules.

Module	Tables
Security	`roles`, `users`

Module	Tables
Academic structure	`academic_years`, `classes`, `sections`, `subjects`, `classrooms`
Profiles	`admins`, `faculty`, `students`
Course allocation	`course_assignments`
Timetable	`periods`, `timetable`
Attendance	`student_attendance`, `faculty_attendance`
Leave	`leave_types`, `leave_requests`
Homework	`homework`, `homework_submissions`
Assignments	`assignments`, `assignment_submissions`
Exams and marks	`exam_types`, `exams`, `exam_schedules`, `grade_scale`, `marks`
Progress reports	`progress_reports`, `progress_report_items`
Notifications	`notifications`

3.1 Key relationships

- Every user belongs to one role: `users.role_id` references `roles`.
- Every person has exactly one profile row, linked by `user_id`: `admins`, `faculty` or `students`.
- A section belongs to a class and an academic year, and has a class teacher (`sections.class_teacher_id` references `faculty`).
- A student belongs to one current section (`students.section_id`).
- `course_assignments` ties together a section, subject, faculty member and classroom.
- `timetable` places a subject, faculty member and classroom into a period on a weekday for a section.
- Attendance, homework, assignments and marks all point back to students or faculty through foreign keys.
- `exams` belong to a class; `exam_schedules` give each subject its date, room and maximum marks; `marks` hold each student's result per scheduled subject.

- `progress_reports` summarise a student's results for one exam, with one `progress_report_items` row per subject.

3.2 Reference IDs in the sample data

Item	IDs
Roles	1 Admin, 2 Faculty, 3 Student
Users	1 admin01; 2-4 faculty (fac_anita, fac_vijay, fac_sara); 5-8 students (stu_arjun, stu_meera, stu_kiran, stu_divya)
Faculty IDs	1 Anita Sharma (Maths), 2 Vijay Reddy (Science), 3 Sara Thomas (English)
Student IDs	1 Arjun, 2 Meera (both Grade 9-A); 3 Kiran, 4 Divya (both Grade 10-A)
Sections	1 Grade 9-A, 2 Grade 10-A
Subjects	1 Mathematics, 2 Science, 3 English
Classrooms	1 Room 101, 2 Room 102, 3 Science Lab
Periods	1 (08:30), 2 (09:15), 3 (10:15)

Sample passwords are placeholders. Replace them with properly hashed passwords (bcrypt or similar) in the application before real use.

4. User Roles and Access

Look up a person and their role:

```
SELECT u.user_id, u.username, r.role_name, u.is_active
FROM users u JOIN roles r ON r.role_id = u.role_id
WHERE u.username = 'stu_arjun';
```

Find the profile ID (student_id or faculty_id) for a username. Most queries below need it:

```
SELECT s.student_id FROM students s JOIN users u ON u.user_id = s.user_id
WHERE u.username = 'stu_arjun';
SELECT f.faculty_id FROM faculty f JOIN users u ON u.user_id = f.user_id
WHERE u.username = 'fac_anita';
```

To disable a person without losing history, set `is_active = FALSE` instead of deleting the user.

5. Table Reference

5.1 Security and structure

Table	Purpose	Key columns
`roles`	Admin, Faculty, Student	role_id (PK), role_name (unique)
`users`	User master for all roles	user_id (PK), role_id (FK), username and email (unique), password_hash, is_active
`academic_years`	School years	year_name (unique), start_date, end_date, is_current
`classes`	Grades such as Grade 9	class_name (unique)
`sections`	A class division such as 10-A	class_id, academic_year_id, class_teacher_id (FKs); unique per class, year and name
`subjects`	Subject master	subject_code (unique), subject_name
`classrooms`	Rooms and labs	room_name (unique), room_type, capacity

5.2 Profiles

Table	Purpose	Key columns
`admins`	Admin profile	user_id (unique FK), designation
`faculty`	Teacher profile	user_id (unique FK), employee_no (unique), qualification, joining_date
`students`	Student profile	user_id (unique FK), admission_no (unique), roll_no, section_id (FK), guardian details

5.3 Teaching and timetable

Table	Purpose	Key columns
`course_assignments`	Who teaches what to which section, and where	academic_year_id, section_id, subject_id, faculty_id, classroom_id; unique per year, section and subject
`periods`	Daily period slots	period_no (unique), start_time, end_time
`timetable`	Weekly schedule	section_id, day_of_week, period_id, subject_id, faculty_id, classroom_id

5.4 Attendance and leave

Table	Purpose	Key columns
`student_attendance`	Attendance per student, date and period	student_id, attendance_date, period_id, subject_id, status, marked_by (faculty)
`faculty_attendance`	Daily faculty attendance	faculty_id, attendance_date, check_in, check_out, status
`leave_types`	Sick, Casual, Earned	type_name, max_days_year

Table	Purpose	Key columns
`leave_requests`	Leave applications and approvals	applicant_id (user), leave_type_id, from_date, to_date, status, reviewed_by (admin user)

5.5 Homework and assignments

Table	Purpose	Key columns
`homework`	Daily homework set by faculty	section_id, subject_id, faculty_id, assigned_date, due_date
`homework_submissions`	One row per student per homework	homework_id, student_id, status, faculty_remarks
`assignments`	Graded assignments and projects	section_id, subject_id, faculty_id, max_marks, due_date
`assignment_submissions`	One row per student per assignment	assignment_id, student_id, marks_obtained, status, feedback

5.6 Exams, marks and reports

Table	Purpose	Key columns
`exam_types`	Unit Test, Mid Term, Final Exam	type_name, weightage
`exams`	An exam event for a class	academic_year_id, exam_type_id, class_id, created_by (admin user)
`exam_schedules`	Subject-wise date, room and marks	exam_id, subject_id, exam_date, classroom_id, invigilator_id, max_marks, pass_marks
`grade_scale`	Percentage to grade mapping	grade, min_percent, max_percent, grade_point
`marks`	Student result per scheduled subject	exam_schedule_id, student_id, marks_obtained, is_absent, grade_id, entered_by (faculty)

Table	Purpose	Key columns
`progress_reports`	One summary per student per exam	student_id, exam_id, total_obtained, percentage, grade_id, attendance_pct, class_rank, is_published
`progress_report_items`	Subject lines of a report	report_id, subject_id, marks_obtained, grade_id
`notifications`	Messages to users	user_id, title, message, is_read

6. Admin Guide

Admins set up the school, then monitor it. Examples below use the sample IDs; change them for your data.

6.1 Add a new student (user and profile in one statement)

```
WITH new_user AS (
  INSERT INTO users (role_id, username, password_hash, first_name, last_name,
email, phone)
  VALUES (3, 'stu_rahul', '<hashed password>', 'Rahul', 'Verma',
'rahul@student.school.edu', '9100000005')
  RETURNING user_id
)
INSERT INTO students (user_id, admission_no, roll_no, section_id,
date_of_birth, gender, guardian_name, guardian_phone, admission_date)
SELECT user_id, 'ADM2026005', 3, 1, '2011-05-10', 'Male', 'Anil Verma',
'9200000005', '2026-06-01'
FROM new_user;
```

To add a faculty member, use role_id 2 and insert into `faculty` instead (employee_no, designation, qualification, joining_date).

6.2 Set up classes, subjects, rooms and sections

```
INSERT INTO classes (class_name) VALUES ('Grade 8');
INSERT INTO subjects (subject_code, subject_name) VALUES ('SOC', 'Social
Studies');
INSERT INTO classrooms (room_name, room_type, capacity) VALUES ('Room 103',
'Classroom', 40);
INSERT INTO sections (class_id, academic_year_id, section_name,
class_teacher_id) VALUES (1, 1, 'B', 3);
```

6.3 Assign a faculty member to a subject, section and classroom

```
INSERT INTO course_assignments (academic_year_id, section_id, subject_id,
faculty_id, classroom_id, periods_per_week)
VALUES (1, 2, 3, 3, 2, 5);
```

6.4 Build the timetable

Insert one row per period. The database blocks clashes automatically: a section, a teacher or a room cannot be used twice in the same period on the same day.

```
INSERT INTO timetable (section_id, day_of_week, period_id, subject_id,
faculty_id, classroom_id)
VALUES (2, 'Tuesday', 1, 1, 1, 2);
```

View the timetable of one section:

```
SELECT day_of_week, period_no, start_time, subject_name, faculty_name,
room_name
FROM v_timetable
WHERE section = 'Grade 10-A'
ORDER BY CASE day_of_week WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2 WHEN
'Wednesday' THEN 3
  WHEN 'Thursday' THEN 4 WHEN 'Friday' THEN 5 ELSE 6 END, period_no;
```

6.4a Attendance reports

Student attendance percentage for everyone, lowest first:

```
SELECT * FROM v_student_attendance_report ORDER BY attendance_pct;
```

Students below 75 percent:

```
SELECT * FROM v_student_attendance_report WHERE attendance_pct < 75;
```

Attendance for one section on one date:

```
SELECT u.first_name, u.last_name, p.period_no, sub.subject_name, sa.status
FROM student_attendance sa
JOIN students s ON s.student_id = sa.student_id
JOIN users u ON u.user_id = s.user_id
JOIN periods p ON p.period_id = sa.period_id
JOIN subjects sub ON sub.subject_id = sa.subject_id
WHERE s.section_id = 2 AND sa.attendance_date = '2026-11-02'
ORDER BY u.first_name, p.period_no;
```

Faculty attendance summary:

```
SELECT f.employee_no, u.first_name, u.last_name, fa.status, COUNT(*) AS days
FROM faculty_attendance fa
JOIN faculty f ON f.faculty_id = fa.faculty_id
JOIN users u ON u.user_id = f.user_id
GROUP BY f.employee_no, u.first_name, u.last_name, fa.status
ORDER BY f.employee_no;
```

6.5 Leave approvals

See pending requests from students and faculty:

```
SELECT * FROM v_pending_leaves;
```

Approve or reject (the admin user_id goes in `reviewed_by`):

```
UPDATE leave_requests
SET status = 'Approved', reviewed_by = 1, reviewed_on = NOW(), admin_remarks
= 'Approved. Get well soon.'
WHERE leave_id = 2;
```

Use status `Rejected` and a remark to reject a request.

6.6 Schedule an exam

```
INSERT INTO exams (academic_year_id, exam_type_id, class_id, exam_name,
start_date, end_date, created_by)
VALUES (1, 3, 2, 'Grade 10 Final Exam 2027', '2027-03-10', '2027-03-15', 1);

INSERT INTO exam_schedules (exam_id, subject_id, exam_date, start_time,
end_time, classroom_id, invigilator_id, max_marks, pass_marks)
VALUES (2, 1, '2027-03-10', '09:00', '12:00', 2, 2, 100, 35);
```

The `exam_id` of the new exam can be found with `SELECT exam_id, exam_name FROM exams;`.

6.7 Exam reports

Student-wise results:

```
SELECT * FROM v_exam_report ORDER BY exam_name, student_name, subject_name;
```

Pass and fail count per subject:

```
SELECT subject_name,
       SUM(CASE WHEN result = 'Pass' THEN 1 ELSE 0 END) AS passed,
       SUM(CASE WHEN result = 'Fail' THEN 1 ELSE 0 END) AS failed
FROM v_exam_report
GROUP BY subject_name;
```

6.8 Progress reports

View published reports with grade and rank:

```
SELECT u.first_name, u.last_name, e.exam_name, pr.total_obtained,
pr.total_max_marks,
       pr.percentage, g.grade, pr.attendance_pct, pr.class_rank,
pr.teacher_remarks
FROM progress_reports pr
JOIN students s ON s.student_id = pr.student_id
JOIN users u ON u.user_id = s.user_id
JOIN exams e ON e.exam_id = pr.exam_id
LEFT JOIN grade_scale g ON g.grade_id = pr.grade_id
ORDER BY e.exam_name, pr.class_rank;
```

Generate reports for a new exam once all marks are entered (replace 2 with the exam_id and 1 with the admin user_id):

```
INSERT INTO progress_reports (student_id, exam_id, total_max_marks,
total_obtained, percentage, generated_by)
SELECT m.student_id, es.exam_id, SUM(es.max_marks), SUM(m.marks_obtained),
       ROUND(100 * SUM(m.marks_obtained) / SUM(es.max_marks), 2), 1
FROM marks m JOIN exam_schedules es ON es.exam_schedule_id =
m.exam_schedule_id
WHERE es.exam_id = 2
GROUP BY m.student_id, es.exam_id;

UPDATE progress_reports pr
SET grade_id = (SELECT g.grade_id FROM grade_scale g WHERE pr.percentage
BETWEEN g.min_percent AND g.max_percent)
WHERE pr.exam_id = 2;

UPDATE progress_reports pr SET class_rank = r.rk
FROM (SELECT report_id, RANK() OVER (ORDER BY percentage DESC) AS rk
      FROM progress_reports WHERE exam_id = 2) r
WHERE pr.report_id = r.report_id;

UPDATE progress_reports SET is_published = TRUE WHERE exam_id = 2;
```

Also insert the subject lines into `progress_report_items` (one row per subject per report) so students can see the subject-wise breakdown.

7. Faculty Guide

Replace `1` with your own `faculty_id` (see Section 4).

7.1 My assigned courses and classrooms

```
SELECT c.class_name, sec.section_name, sub.subject_name, cr.room_name,
ca.periods_per_week
FROM course_assignments ca
JOIN sections sec ON sec.section_id = ca.section_id
JOIN classes c ON c.class_id = sec.class_id
JOIN subjects sub ON sub.subject_id = ca.subject_id
JOIN classrooms cr ON cr.classroom_id = ca.classroom_id
WHERE ca.faculty_id = 1;
```

7.2 My timetable

```
SELECT t.day_of_week, p.period_no, p.start_time, p.end_time, c.class_name,
sec.section_name, sub.subject_name, cr.room_name
FROM timetable t
JOIN periods p ON p.period_id = t.period_id
JOIN sections sec ON sec.section_id = t.section_id
JOIN classes c ON c.class_id = sec.class_id
JOIN subjects sub ON sub.subject_id = t.subject_id
JOIN classrooms cr ON cr.classroom_id = t.classroom_id
WHERE t.faculty_id = 1
ORDER BY CASE t.day_of_week WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2 WHEN
'Wednesday' THEN 3
  WHEN 'Thursday' THEN 4 WHEN 'Friday' THEN 5 ELSE 6 END, p.period_no;
```

7.3 Mark student attendance

Status values: `Present`, `Absent`, `Late`, `Excused`. One row per student, date and period.

```
INSERT INTO student_attendance (student_id, attendance_date, period_id,
subject_id, status, marked_by, remarks)
VALUES (1, '2026-11-03', 1, 1, 'Present', 1, NULL),
       (2, '2026-11-03', 1, 1, 'Absent', 1, 'Fever');
```

To correct a mistake:

```
UPDATE student_attendance SET status = 'Late'
WHERE student_id = 2 AND attendance_date = '2026-11-03' AND period_id = 1;
```

7.4 My own attendance

```
INSERT INTO faculty_attendance (faculty_id, attendance_date, check_in,
status)
VALUES (1, '2026-11-03', '08:20', 'Present');

UPDATE faculty_attendance SET check_out = '16:00'
WHERE faculty_id = 1 AND attendance_date = '2026-11-03';
```

7.5 Assign homework

```
INSERT INTO homework (section_id, subject_id, faculty_id, title, description,
assigned_date, due_date)
VALUES (1, 1, 1, 'Algebra Worksheet 6', 'Solve questions 1-15', '2026-11-03',
'2026-11-06');
```

Create a pending submission row for every student in the section so you can track who has not submitted:

```
INSERT INTO homework_submissions (homework_id, student_id)
SELECT 3, s.student_id FROM students s WHERE s.section_id = 1;
```

Use the correct `homework_id` (find it with `SELECT homework_id, title FROM homework;`). Check who has not submitted and record your review:

```
SELECT u.first_name, u.last_name, hs.status
FROM homework_submissions hs
JOIN students s ON s.student_id = hs.student_id
JOIN users u ON u.user_id = s.user_id
WHERE hs.homework_id = 1 AND hs.status = 'Pending';

UPDATE homework_submissions SET status = 'Checked', faculty_remarks = 'Well
done'
WHERE homework_id = 1 AND student_id = 1;
```

7.6 Assignments and grading

```
INSERT INTO assignments (section_id, subject_id, faculty_id, title,
description, max_marks, assigned_date, due_date)
VALUES (2, 2, 2, 'Lab Report: Acids and Bases', 'Submit a written report',
20, '2026-11-03', '2026-11-20');

UPDATE assignment_submissions
SET marks_obtained = 16.5, status = 'Graded', feedback = 'Clear observations'
WHERE assignment_id = 1 AND student_id = 4;
```

7.7 Enter exam marks

The grade is looked up from `grade_scale`. This example is for an exam where the maximum is 100, so marks equal the percentage.

```
INSERT INTO marks (exam_schedule_id, student_id, marks_obtained, is_absent,
grade_id, entered_by)
VALUES (4, 3, 85, FALSE,
        (SELECT grade_id FROM grade_scale WHERE 85 BETWEEN min_percent AND
max_percent), 1);
```

For an absent student, set `is_absent` to TRUE and `marks_obtained` to NULL. Use your own `exam_schedule_id` from `SELECT * FROM exam_schedules;`. Marks can be viewed with:

```
SELECT * FROM v_exam_report WHERE subject_name = 'Mathematics';
```

7.8 Apply for leave

```
INSERT INTO leave_requests (applicant_id, leave_type_id, from_date, to_date,
reason)
VALUES (2, 2, '2026-11-20', '2026-11-20', 'Personal work');
```

Here `applicant_id` is your `user_id` (not your faculty_id). The request stays `Pending` until an admin reviews it. Check its status:

```
SELECT leave_id, from_date, to_date, status, admin_remarks FROM
leave_requests WHERE applicant_id = 2;
```

8. Student Guide

The examples use the username `stu_arjun`. Replace it with your own username.

8.1 My profile

```
SELECT s.admission_no, s.roll_no, u.first_name, u.last_name, c.class_name,
sec.section_name
FROM students s
JOIN users u ON u.user_id = s.user_id
JOIN sections sec ON sec.section_id = s.section_id
JOIN classes c ON c.class_id = sec.class_id
WHERE u.username = 'stu_arjun';
```

8.2 My course details

```
SELECT sub.subject_name, fu.first_name || ' ' || fu.last_name AS teacher,
cr.room_name
FROM students s
JOIN users su ON su.user_id = s.user_id
JOIN course_assignments ca ON ca.section_id = s.section_id
JOIN subjects sub ON sub.subject_id = ca.subject_id
JOIN faculty f ON f.faculty_id = ca.faculty_id
JOIN users fu ON fu.user_id = f.user_id
JOIN classrooms cr ON cr.classroom_id = ca.classroom_id
WHERE su.username = 'stu_arjun';
```

8.3 My timetable

```
SELECT t.day_of_week, t.period_no, t.start_time, t.end_time, t.subject_name,
t.faculty_name, t.room_name
FROM v_timetable t
JOIN students s ON s.section_id = t.section_id
JOIN users u ON u.user_id = s.user_id
WHERE u.username = 'stu_arjun'
ORDER BY CASE t.day_of_week WHEN 'Monday' THEN 1 WHEN 'Tuesday' THEN 2 WHEN
'Wednesday' THEN 3
  WHEN 'Thursday' THEN 4 WHEN 'Friday' THEN 5 ELSE 6 END, t.period_no;
```

8.4 My attendance

Subject-wise attendance with percentage:

```
SELECT sub.subject_name, COUNT(*) AS total_periods,
       SUM(CASE WHEN sa.status IN ('Present','Late') THEN 1 ELSE 0 END) AS
attended,
       ROUND(100.0 * SUM(CASE WHEN sa.status IN ('Present','Late') THEN 1
ELSE 0 END) / COUNT(*), 2) AS pct
FROM student_attendance sa
JOIN students s ON s.student_id = sa.student_id
JOIN users u ON u.user_id = s.user_id
JOIN subjects sub ON sub.subject_id = sa.subject_id
WHERE u.username = 'stu_arjun'
GROUP BY sub.subject_name;
```

8.5 My homework and assignments

Homework with due dates and status:

```
SELECT sub.subject_name, h.title, h.due_date, hs.status, hs.faculty_remarks
FROM homework_submissions hs
JOIN homework h ON h.homework_id = hs.homework_id
JOIN subjects sub ON sub.subject_id = h.subject_id
JOIN students s ON s.student_id = hs.student_id
JOIN users u ON u.user_id = s.user_id
WHERE u.username = 'stu_arjun'
ORDER BY h.due_date;
```

Assignments with marks and feedback:

```
SELECT sub.subject_name, a.title, a.due_date, a.max_marks,
asub.marks_obtained, asub.status, asub.feedback
FROM assignment_submissions asub
JOIN assignments a ON a.assignment_id = asub.assignment_id
JOIN subjects sub ON sub.subject_id = a.subject_id
JOIN students s ON s.student_id = asub.student_id
JOIN users u ON u.user_id = s.user_id
WHERE u.username = 'stu_arjun';
```

8.6 Exam schedule and marks

Upcoming exam schedule for my class:

```
SELECT e.exam_name, sub.subject_name, es.exam_date, es.start_time,
es.end_time, cr.room_name, es.max_marks
FROM students s
JOIN users u ON u.user_id = s.user_id
JOIN sections sec ON sec.section_id = s.section_id
JOIN exams e ON e.class_id = sec.class_id
JOIN exam_schedules es ON es.exam_id = e.exam_id
JOIN subjects sub ON sub.subject_id = es.subject_id
JOIN classrooms cr ON cr.classroom_id = es.classroom_id
WHERE u.username = 'stu_kiran'
ORDER BY es.exam_date;
```

My marks:

```
SELECT exam_name, subject_name, max_marks, marks_obtained, result, grade
FROM v_exam_report WHERE student_name = 'Kiran Patel';
```

8.6a My progress report

```
SELECT e.exam_name, sub.subject_name, pri.marks_obtained, pri.max_marks,
g.grade
FROM progress_reports pr
JOIN progress_report_items pri ON pri.report_id = pr.report_id
JOIN subjects sub ON sub.subject_id = pri.subject_id
JOIN exams e ON e.exam_id = pr.exam_id
JOIN students s ON s.student_id = pr.student_id
JOIN users u ON u.user_id = s.user_id
LEFT JOIN grade_scale g ON g.grade_id = pri.grade_id
WHERE u.username = 'stu_kiran' AND pr.is_published = TRUE;
```

Overall result (percentage, grade, attendance, rank, remarks):

```
SELECT pr.percentage, g.grade, pr.attendance_pct, pr.class_rank,
pr.teacher_remarks
FROM progress_reports pr
JOIN students s ON s.student_id = pr.student_id
JOIN users u ON u.user_id = s.user_id
LEFT JOIN grade_scale g ON g.grade_id = pr.grade_id
WHERE u.username = 'stu_kiran' AND pr.is_published = TRUE;
```

8.7 Apply for leave and check notifications

```
INSERT INTO leave_requests (applicant_id, leave_type_id, from_date, to_date,
reason)
VALUES (5, 1, '2026-11-10', '2026-11-11', 'Fever');

SELECT title, message, created_at FROM notifications WHERE user_id = 5 AND
is_read = FALSE;

UPDATE notifications SET is_read = TRUE WHERE user_id = 5;
```

Here `5` is the student's `user_id`.

9. Built-in Reports (Views)

View	Shows	Typical user
`v_student_attendance_report`	Total periods, attended periods and attendance percentage per student	Admin
`v_exam_report`	Exam, student, subject, marks, pass or fail, grade	Admin, Faculty
`v_pending_leaves`	Leave requests waiting for approval	Admin
`v_timetable`	Weekly timetable with section, period times, subject, teacher and room	Everyone

Views always show current data. Query them like tables, for example `SELECT * FROM v_exam_report WHERE result = 'Fail';`.

10. Data Rules and Constraints

The database enforces these rules automatically. An invalid change is rejected with an error message.

Rule	Where it applies
Usernames and emails are unique	`users`
One profile per user	`admins`, `faculty`, `students` (`user_id` is unique)
Admission and employee numbers are unique	`students.admission_no`, `faculty.employee_no`
No timetable clashes	`timetable`: one class per section, teacher and room per day and period
One course assignment per subject per section per year	`course_assignments`
One attendance record per student, date and period	`student_attendance`

Rule	Where it applies
One attendance record per faculty member per day	`faculty_attendance`
One submission per student per homework or assignment	`homework_submissions`, `assignment_submissions`
One mark per student per exam subject	`marks`
One progress report per student per exam	`progress_reports`
Allowed status values	attendance status, leave status, submission status, gender, room type, weekday
Date and time order	end date after start date, period end after start, leave end not before start, due date not before assigned date
Pass marks cannot exceed maximum marks	`exam_schedules`
Capacity must be positive	`classrooms`

10.1 Foreign keys and insert order

A row can only refer to a row that already exists. Insert parent data before child data in this order:

1. roles, academic_years, classes, subjects, classrooms, periods, leave_types, exam_types, grade_scale
2. users
3. admins, faculty
4. sections
5. students
6. course_assignments, timetable
7. attendance, leave_requests, homework, assignments
8. homework_submissions, assignment_submissions
9. exams, exam_schedules
10. marks
11. progress_reports, progress_report_items
12. notifications

Rows cannot be deleted while other rows still refer to them (no cascading deletes are defined). Deactivate users instead (`is_active = FALSE`) to preserve history.

10.2 Indexes

Indexes exist on the foreign keys most used in reports (for example student, section, attendance date, leave status, marks by student). Unique constraints automatically create their own indexes. Run `\d tablename` to see all indexes of a table.

11. Maintenance

11.1 Backup

Run from the operating system command line, not inside psql:

```
pg_dump -U postgres -n school school_db > school_backup.sql
```

Schedule this daily, and keep copies on a different drive or machine.

11.2 Restore

```
psql -U postgres -d school_db -f school_backup.sql
```

Restore into an empty or fresh schema. If the `school` schema already exists with data, drop it first or you will see duplicate object errors.

11.3 Reset to sample data

Re-run `school_management_postgres92.sql` . This deletes all current data in the `school` schema and recreates the sample records. Back up first if you need to keep anything.

11.4 Database logins and permissions

The Admin, Faculty and Student roles are application roles. To also restrict access at database level, create PostgreSQL login roles. Example for a read-only reporting user:

```
CREATE ROLE report_user LOGIN PASSWORD 'ChangeMe#123';
GRANT USAGE ON SCHEMA school TO report_user;
GRANT SELECT ON ALL TABLES IN SCHEMA school TO report_user;
```

Example for an application user that can read and write data:

```
CREATE ROLE school_app LOGIN PASSWORD 'ChangeMe#456';
GRANT USAGE ON SCHEMA school TO school_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA school TO
school_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA school TO school_app;
```

Repeat the grants after adding new tables.

11.5 Security recommendations

- Never store plain-text passwords. Store a bcrypt or argon2 hash in `password_hash`.
- Do not let students or faculty connect directly with the postgres superuser. Use the application layer or limited login roles.
- Student and faculty queries must always filter by the logged-in person (username, student_id or faculty_id) so they only see their own data.
- Take regular backups and test a restore at least once a term.
- PostgreSQL 9.2 no longer receives security updates. Plan an upgrade to a supported version when possible. The script can be used on newer versions as well.

11.6 Year-end routine

1. Insert the new row in `academic_years` and set the old one to `is_current = FALSE`.
2. Create the new year's `sections`.
3. Move students to their new sections: `UPDATE students SET section_id = <new section> WHERE student_id = <id>;`
4. Create new `course_assignments`, `timetable` rows and exams.

Past attendance, marks and reports stay in the database for history.

12. Troubleshooting

Message or problem	Cause	Fix
`relation "users" does not exist`	Search path not set	Run `SET search_path TO school;` or use `school.users`

Message or problem	Cause	Fix
`duplicate key value violates unique constraint`	Row already exists, or a timetable clash	Check the existing row with a SELECT, then UPDATE it or change the values
`violates foreign key constraint`	The referenced parent row does not exist, or you are deleting a row still in use	Insert the parent first (see 10.1), or deactivate instead of deleting
`violates check constraint`	Value not allowed, such as a wrong status spelling or end date before start date	Use the allowed values listed in Section 10 (they are case sensitive)
`null value in column ... violates not-null constraint`	A required column was left out	Supply a value for the column
`syntax error at or near ...`	Typo, or MySQL-style syntax used	Check commas and quotes; use only PostgreSQL syntax
Query returns no rows	Wrong ID, name or filter	Check IDs with Section 3.2, and remember username and text matches are case sensitive
`permission denied for table`	The login role lacks rights	Apply the grants in Section 11.4
`psql: could not connect`	Server not running, or wrong host, port or password	Start the PostgreSQL service and check connection details

12.1 Quick health check

```
SELECT 'users' AS table_name, COUNT(*) FROM users
UNION ALL SELECT 'students', COUNT(*) FROM students
UNION ALL SELECT 'faculty', COUNT(*) FROM faculty
UNION ALL SELECT 'timetable', COUNT(*) FROM timetable
UNION ALL SELECT 'marks', COUNT(*) FROM marks;
```

Sample data returns 8 users, 4 students, 3 faculty, 8 timetable rows and 6 marks.

12.2 Find students without a profile or users without a role profile

```
SELECT u.user_id, u.username FROM users u
WHERE u.role_id = 3 AND NOT EXISTS (SELECT 1 FROM students s WHERE s.user_id
= u.user_id);
```

13. Quick Reference

I want to...	Who	Use
Add a user	Admin	Section 6.1
Assign a teacher to a subject	Admin	Section 6.3
Create the timetable	Admin	Section 6.4
See attendance reports	Admin	`v_student_attendance_report`, Section 6.4a
Approve leave	Admin	`v_pending_leaves`, Section 6.5
Schedule an exam	Admin	Section 6.6
Generate progress reports	Admin	Section 6.8
See my classes and rooms	Faculty	Sections 7.1 and 7.2
Mark attendance	Faculty	Section 7.3
Give homework or assignments	Faculty	Sections 7.5 and 7.6
Enter marks	Faculty	Section 7.7
See my timetable	Faculty, Student	Sections 7.2 and 8.3
Check my attendance	Student	Section 8.4
See homework and marks	Student	Sections 8.5 and 8.6
See my progress report	Student	Section 8.6a